Automated Testing of the GUI of a Real-Life Engineering Software using Large Language Models*

Tim Rosenbach[1], David Heidrich[2], and Alexander Weinert[1]

[1] German Aerospace Center (DLR), Institute of Software Technology, Cologne, Germany
{tim.rosenbach, alexander.weinert}@dlr.de
[2] German Aerospace Center (DLR), Institute of Software Technology, Oberpfaffenhofen, Germany
david.heidrich@dlr.de

Abstract. One important step in software development is testing the finished product with actual users. These tests aim, among other goals, at determining unintuitive behavior of the software as it is presented to the end-user. Moreover, they aim to determine inconsistencies in the user-facing interface. They provide valuable feedback for the development of the software, but are time-intensive to conduct.
In this work, we present GERALLT, a system that uses Large Language Models (LLMs) to perform exploratory tests of the Graphical User Interface (GUI) of a real-life engineering software. GERALLT automatically generates a list of potential unintuitive and inconsistent parts of the interface. We present the architecture of GERALLT and evaluate it on a real-world use case of the engineering software, which has been extensively tested by developers and users. Our results show that GERALLT is able to determine issues with the interface that support the software development team in future development of the software.

Keywords: Software Testing · Graphical User Interface · Acceptance Testing · Large Language Models

1 Introduction

When developing software, large efforts are spent on quality assurance [2], which is mainly performed by conducting a number of automated and manual tests on the software. Developers use such tests to determine the compliance of the software with domain requirements as well as to determine the quality of the software. Popular qualitative metrics for the software under development are, e.g., its performance, security, or accessibility [3]. To varying degrees, such tests can be automated and be executed and evaluated without human input.

One major aspect of software quality, which is not easily captured via automated tests, is the acceptance of the software by end users. User acceptance depends not only on the satisfaction of domain requirements, but also, among others, on the intuitiveness and consistency of the user interface. These metrics are hard to capture using automated tests. Hence, these criteria are often tested for using manual tests of the interface of the software. While manual tests are perceived as effective by software developers, they are also expensive in terms of time and resources spent [4]. Although some work exists on the automation of such tests, that work is mainly focused on either web-based software, or applications for popular mobile operating systems, like Android. [5,6,7,8,9,10,11]

In this work, we present GERALLT, a system that aims to support developers in testing the interface of a real-life desktop-based engineering software that is in productive use throughout numerous institutes of the German Aerospace Center (DLR). At the core of GERALLT are two components based on Large Language Models (LLMs), a type of generative Artificial Intelligence (AI) capable of producing human-like text based on patterns learned from vast amounts of data. One of these components explores the Graphical User Interface (GUI), while the other evaluates the observed behavior for unintuitive or inconsistent interactions, as well as for functional errors. We then evaluate the issues determined by GERALLT via discussion with the developers of the software.

The main contribution of this work consists of an architecture description of GERALLT and its evaluation. GERALLT comprises multiple components and is partially based on previous work due to Lui et al. [5]. We evaluate GERALLT by testing a specific feature of the engineering software with it and discussing the issues GERALLT determined with the software engineers.

The remainder of this work is organized as follows: After giving an overview over previous work in Section 2 we describe the system under test, its use at DLR, and its technical implementation in Section 4. Afterwards, we

* This work is based on the bachelor thesis of the first author. [1]

evaluate GERALLT in Section 5 and conclude this work with a summary of our results and perspectives for future work in Section 6.

2 Background and Related Work

LLMs are a class of generative AI, which are trained on large amounts of textual data and refined through human feedback. Given a textual input prompt, they produce textual responses, which mimic human responses with surprising accuracy in a diverse array of tasks. In recent years, there has been a flurry of research on new domains in which LLMs can be applied. [12] In particular, there exist a number of works on the application of LLMs in software quality assurance. Among others, these include using LLMs for generating [13,10,11] and executing [7,14] test cases, for exploring the system under test, e.g., via fuzzing [9], and for repairing detected issues [15,16,17].

Moreover, there exist numerous works on the automation of GUI testing [18]. Of particular interest are novel strategies for the traversal of applications for the Android operating system [19,20] as well as the use of AI for the traversal of arbitrary GUIs [8,6]. In the remainder of this section, we focus on three pieces of related work that align most closely with our work presented here.

Liu et al. [5] developed GPT Droid, an LLM-based system for testing Android apps. GPT Droid extracts the GUI context from an app and encodes it into prompts for the LLM. The LLM generates operational commands, which GPT Droid translates into executable actions. GPT Droid includes a functionality-aware memory mechanism to retain long-term knowledge of the testing process. This mechanism allows the LLM to guide exploration based on app functionality. In evaluations on 93 Android apps, GPT Droid achieved 75% activity coverage and detected 31% more bugs than the best baseline. It also identified 53 new bugs on Google Play, with developers confirming or fixing 35 of them.

Gao et al. [21] developed a system for automating desktop GUI tasks by utilizing LLMs. The system includes a GUI Parser that converts screenshots and metadata into a structured representation by combining Optical Character Recognition (OCR), icon detection, and other vision tools. This representation enables the system to understand diverse UI elements and their spatial relationships. Their experiments showed the framework achieving a 46% success rate, revealing the challenges and potential for improvements in this domain.

Zimmermann and Koziolek [6] use LLMs to test a web-based example application. Web-based applications have a natural textual representation, which is rendered by the browser. In contrast, we investigate a desktop-based application where we first need to generate a textual representation of the current state of the GUI. Moreover, here we investigate the capabilities of LLMs for testing a real-world application instead of an example application specifically constructed for the purpose of testing.

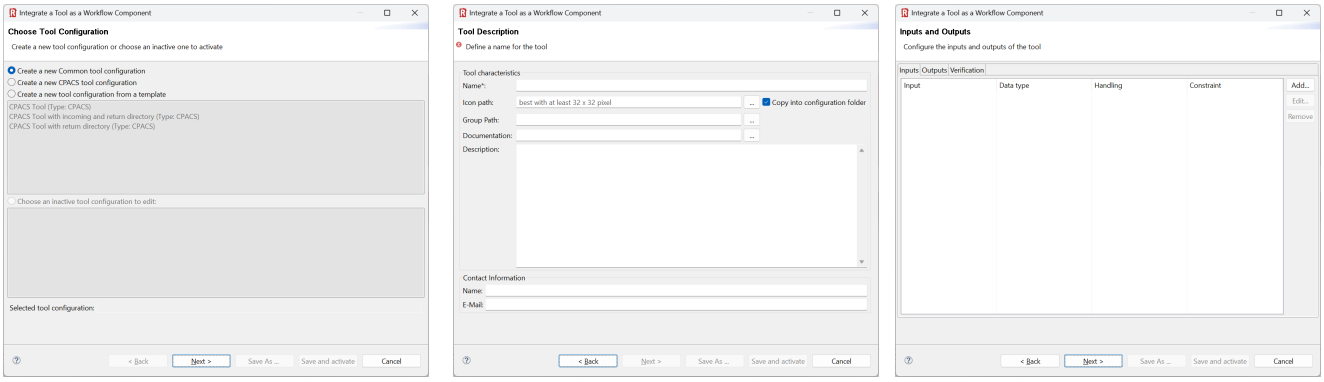
Our work builds upon the approaches of Liu et al. and Gao et al.. We use the concept of Liu et al. for automated GUI testing and combine it with the approach of Gao et al. for the automation of a desktop application. The key novelty of our work lies in automating GUI testing for a desktop application. While prior research has focused on mobile and web applications, our approach extends LLM-based testing techniques to a real-world desktop environment.

3 The Use Case

RCE is an open-source software used for designing, implementing, and executing simulation toolchains. Engineers at DLR use RCE as part of their daily work to simulate complex systems, such as aircraft, ships, and satellites [22,23]. RCE itself does not provide any discipline-specific simulations "out of the box" but instead relies on engineers *integrating* their pre-existing simulations into RCE. Integrating a simulation into RCE mainly amounts to defining inputs and outputs of the simulation as well as providing a shell script defining the invocation of the simulation. While straightforward in concept, there are numerous additional decisions to be made by the tool integrator, such as defining correct paths for execution, setting up the environment of the simulation, or allowing for concurrent executions of simulations. In practice, the integration of an existing simulation is a complex task that requires in-depth knowledge of both RCE and the simulation to be integrated.

To simplify the integration, RCE provides users with a desktop-based GUI that guides the integrator through the integration. This GUI consists of multiple pages of Eclipse Rich Client Platform (RCP)-based forms which ask the user to supply the information required for integration, sometimes containing additional pop-up windows for, e.g., specifying the inputs and outputs. We illustrate the pages of this wizard in Figure 1. Keeping with the typical vernacular of desktop-based software, we call this GUI the *tool integration wizard*.

Since RCE is used in numerous research long-running projects stability is of paramount importance. Hence, the developers pay particular attention to quality assurance. This takes the form not only of automated unit tests, integration tests, and end-to-end tests, but also of manual



(a) First Page (b) Second Page (c) Third Page

(d) Fourth Page (e) Fifth Page (f) Sixth Page

Fig. 1: Pages of the tool integration wizard of RCE

exploratory GUI tests. [24]. The automated tests serve mainly to protect against regressions of RCE's functionality. In contrast, the manual tests aim to uncover unintuitive and overly complex GUI behavior, which is hard to codify using classical testing setups.

Manual tests, while effective, require significant labor investment. A typical interactive testing session occupies around five to seven software engineers over the course of one to three weeks, depending on the agreed-upon testing scope. [24] Hence, even partial automation of this process promises to substantially improve the development process of RCE. In the following section, we describe an architecture for a software tool that aims to augment the manual testing process.

4 Testing RCE with LLMs

To test RCE we have developed GERALLT, a system focusing on using two LLM-based agents. This system is based on previous work by Liu et al. [5] and Gao et al. [21]. GERALLT includes a GUI Parser that converts screenshots and metadata into a structured representation by combining OCR, icon detection, and other vision tools. The aim of one agent is to control RCE, while the aim of the other one is to observe the evolution of the GUI of RCE and to inform the user about observed inconsistencies. After giving an overview over the architecture of GERALLT and its constituent components in Section 4.1 we describe the prompts used for the two LLM-based components in Section 4.2 and in Section 4.3, respectively.

4.1 System Architecture

We present the architecture of GERALLT in Figure 2. The aim of GERALLT is to mimic the behavior of a human tester, i.e., to execute a loosely defined task with RCE and to provoke unintuitive behavior while doing so. These loosely defined task only define a goal and not concrete steps to achieve it. Moreover, the human tester is given no guidance except for the existing documentation and their personal intuition. We separate the two tasks of controlling RCE and of determining unintuitive behavior into two LLM-based components, which we call the *controller* and the *evaluator*, respectively. The controller is responsible for executing meaningful actions on the GUI The evaluator checks the GUI for issues after each action are performed.

On startup, GERALLT takes a task as input and initializes an instance of RCE. In each iteration, GERALLT constructs a prompt for the controller comprised of a) the task originally given to GERALLT, b) a structured description of the current state of the GUI produced by a GUI Parser, c) a list of actions possible on the GUI widgets available in the current state, d) documentation given by RCE, e) a screenshot of the current state of the GUI (if the LLM can interpret images), and f) the actions previously taken by the controller. We describe the construction of this prompt in more detail in Section 4.2. GERALLT gives this prompt to the controller, which outputs a selection of one of the possible actions. Afterwards, GERALLT parses the output of the controller and tries to execute the selected action on the RCE instance. It moreover records the attempted action as well as whether or not it was successfully executed in an action log.

After the execution was attempted, GERALLT constructs a prompt for the evaluator comprised of a) a screenshot of the GUI before the action was attempted, b) a screenshot of the GUI after the action was attempted, and c) a description of the attempted action. We describe the construction of this prompt in more detail in Section 4.3. The prompt moreover contains a request to determine unintuitive behavior of the GUI.

4.2 Controller Prompt

In this section we describe the prompt given to the controller agent. We showcase an instance of this prompt in Table 1. The overall structure of the prompt follows a classical pattern as described by Peckham, Jeff, et al. [25]. We first provide relevant context about the role of the LLM as well as a concrete description of its task. Afterwards, we give examples of the desired interactions as well as a history of previous user interactions. Finally, we conclude the prompt by concisely reiterating the task.

In the first section of the prompt, we provide context about the role of the LLM. For this, we describe that the LLM will receive a concrete task and textual information about the current state of the GUI of RCE and that it is expected to pick to next action to perform on the GUI in order to achieve its stated task. Recall that we aim to have GERALLT simulate the behavior of a new and non-expert user of RCE. Hence, we omit descriptions of the intended use of RCE as well as of its functionality on purpose.

Having described the context of GERALLT, we state its concrete task, namely to "act as a GUI-Tester for the software RCE". We moreover provide the evaluation criterion of covering "as many different GUI-elements as possible". Again, we consciously omit more concrete definitions of steps to take. This serves having GERALLT

Table 1: Composition of the controller prompt. Repetitions omitted and denoted by ellipses. Newlines omitted where possible.

Part	Example
Role	Your Role: You are an automated system that controls the software RCE. You are given a task, the GUI of RCE and documentation about the software in textual form. You have to interact with the GUI to achieve the given task. You can control the GUI by sending actions to the software. The software will execute the actions and give you feedback about the result, by sending you the new state of the GUI and whether the action was successful or not. You must use the information about the GUI and the documentation to decide which actions to take. Include the information about the position of the GUI-Elements and the text of the GUI-Elements in your decision. Also consider which Parent-Elements the GUI-Elements have. It is important to take the context of the GUI-Elements into account when deciding which actions to take You can also use the feedback about the result of the actions to decide which actions to take next.
Task	Your task: Act as a GUI-Tester for the software RCE. Cover all pages of the Tool Integration Wizard. Use as many different UI-Elements as possible.
Documentation	Documentation: # CPACS Tool Properties (optional) ## Synopsis Configure the CPACS tool specific values. ## Usage Fill in the following values to make your tool using the CPACS specific features, e.g.input and output mapping. - **Incoming CPACS endpoint name**: Select theinput that represents the incoming CPACS file. This input must be configuredon the "Inputs and Outputs" page. **Note** The data type of this input must be "File". Usage must be "required". - **Input mapping file**: Select or enter the mapping file for input mapping. Supported file extensions are ".xml" for classic mapping and ".xsl" for advanced XSLT-mapping. **Note** The path must be relative to the tool directory configured on the "Launch Settings" page. The file chooser dialog regards thatfact. [...]
GUI	The current state of the GUI: { "class_name": "Dialog", "control_type": "WindowSpecification", "control_id": 0, "rectangle": ["L4429", "T1655", "R5172", "B2299"], "text": "Integrate a Tool as a Workflow Component", "sub_elements": [...] }
Action	There are different types of GUI-Elements in the GUI of RCE. UIAWrapper and StaticWrapper are GUI-Elements that can not be interacted with. Their only purpose is to display information or group other Gui Elements. The ButtonWrapper is a GUI-Element that can be clicked. ... To control a GUI-Element output a command in the following Format: ⟨action ⟩(⟨control id⟩), for example click(134478) For each control type there are different actions posible. The StaticWrapper, ToolbarWrapper and UIAWrapper have no actions. The ButtonWrapper has the click(⟨control_id⟩) action, for example click(134478) ... You must format your output in JSON as the following: "action": "⟨action⟩", "explanation": "⟨what the action does and why you do it⟩" example 1: "action": "click(134478)", "explanation": "click next to get to the second page" ...
Action Log	The previous actions: ["action": "write(2166788, 'Number of Threads')", "explanation": "Enter a display name 'Number of Threads' for the 'numThreads' property to make the key human-readable and provide context in the properties view.", "status": "executed" , ...]
Closing	What action do you want to take to do the next step for achieving the given task?

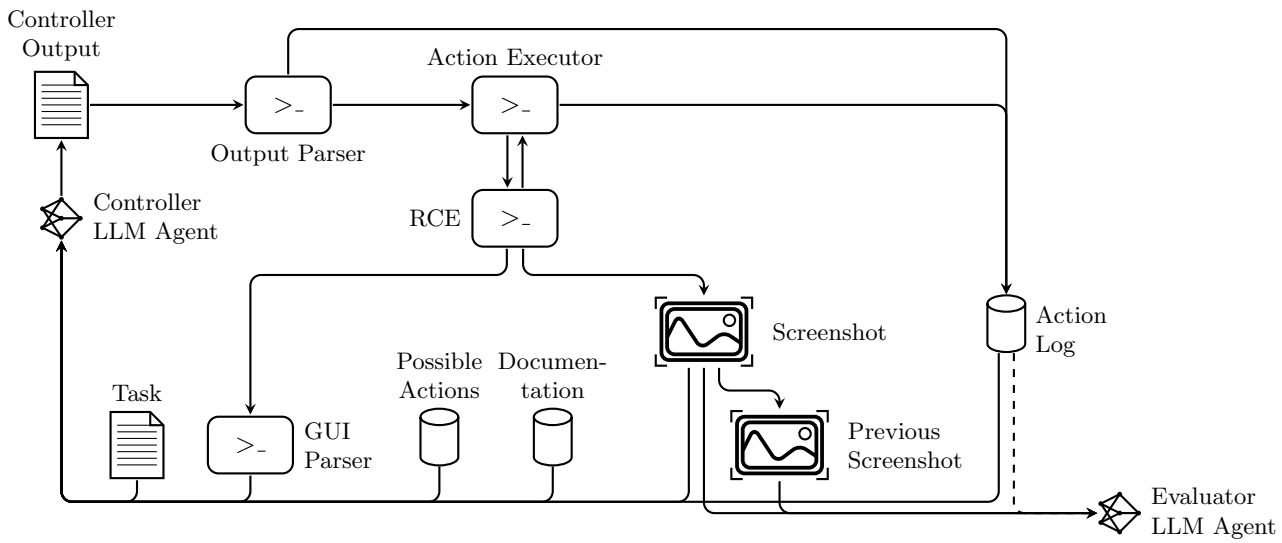


Fig. 2: Architecture of GERALLT. The component "Previous Screenshot" holds the screenshot of the GUI taken during the last iteration. It is replaced by an updated screenshot after each iteration. The dashed line in the bottom right denotes that the evaluator only receives the last action performed on the GUI instead of the complete log.

emulate the manual testing, which aims to explore all possible interactions of GUI elements.

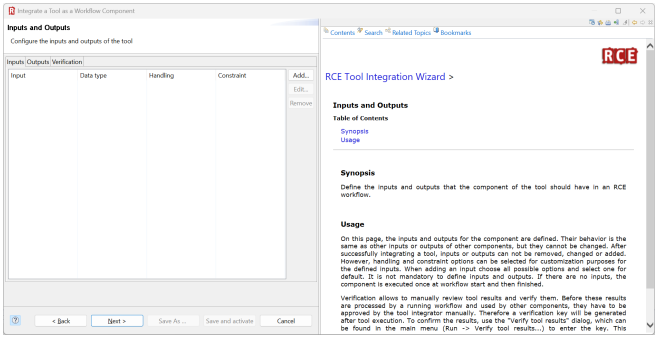


Fig. 3: The appearance of the online help in the tool integration of RCE.

Initial experiments with LLMs showed that the LLM needs more information about RCE other than the GUI information. This manifested in the controller agent initially failing to proceed through the various stages of the tool integration. This, in turn, was mainly due to to the text emitted by the LLM to the GUI not clearing input sanitization, e.g., not providing a well-defined path when requested. To overcome this obstacle, we include relevant parts of the RCE documentation in the prompt. This documentation is included in RCE and also available to human testers. We illustrate the appearance of this online help in the right-hand side of Figure 3.

Afterwards, we provide the LLM with a textual description of the current state of the GUI. This description is generated by the *GUI Parser*. The GUI Parser uses the PyWinAuto Python package [26] to extract the GUI elements on Operating System (OS) level. This works similar to the approach of Liu et al. [5], but instead of Android the OS is Windows. The description takes the form of a JSON representation of the widgets comprising the GUI. As each widget can contain multiple child-widgets, this representation describes a tree of widgets. Each widget is described by its name, its type (e.g., a static text, a combo box, or a text input field), its unique ID, its position on the screen. Additionally, some widgets have additional information, e.g. a checkbox has the information if it is checked.

This concludes the description of the current state of the system under control. It remains to define the actions available to the LLM. To this end, we define for each type of element a list of possible actions. Moreover, we state that the output of the LLM shall conform to a given JSON schema to simplify subsequent parsing. In addition, we provide a list of previously taken actions to make the evolution of the GUI up to this point explicit. This way, we do not have to rely on the context window of the LLM to retain the previously taken actions.

Finally, we conclude the prompt with a concrete question about the next action to be executed on the GUI.

This concludes the description of the prompt for the controller LLM and the rationale behind its construction. In the next section we describe the prompt for the evaluator LLM.

4.3 Evaluator Prompt

The aim of the evaluator is to "observe" the effect of the controller's actions on the GUI of RCE and to determine whether any actions prescribed by the controller have an unintuitive consequence on the GUI. For this, the evaluator receives as input a screenshot from before the action and one after the action together with a textual prompt. The prompt again consists of an initial section describing the context and concrete task of the component. Similarly to the documentation of RCE provided to the controller, the task description contains a list of common errors and unintuitive behaviors typically caught by human testers. The prompt then contains instructions on formatting the output as well as the most recent action prescribed by the controller. This description is taken directly from the explanation produced by the controller.

Since the prompt for the evaluator follows a similar structure and analogous reasoning to the controller prompt, we omit a detailed description. Instead, we provide an instance of this prompt in Table 2. This concludes the description of the architecture of GERALLT. In the following section, we evaluate GERALLT and discuss its known limitations.

5 Evaluation and Known Limitations

To evaluate GERALLT, we implemented the individual components described in Figure 2.[27] We implemented most components of GERALLT as Python scripts, namely the Output Parser, the Action Executor and the construction of the controller prompt and evaluator prompt. For the implementation of the controller agent and evaluator agent, previous work by Rosenbach [1] determined ChatGPT by OpenAI [28] to be the most promising LLM for this use case. So we use the GPT-4o model over OpenAI's API, so that GERALLT runs without human intervention. Since ChatGPT is multimodal, we are able to supply a screenshot of the current state of the GUI to the LLM in addition to the textual prompt, as described in Section 4.2. GPT's responses are very variable. However, this is beneficial for covering more parts of the tested software application. Moreover, we used PyWinAuto [29] to implement the GUI Parser and the Action Executor.

Recall that it was our aim to construct a tool that supports software developers in assuring the quality of RCE. In particular, we aimed to support developers in performing time-intensive exploratory GUI tests. These tests are targeted at finding "unintuitive" or "unnatural" behavior of the GUI of RCE as well as finding functional errors. Hence, there exists no baseline of existing or known errors against we can evaluate the results of our tool. Instead, we performed a qualitative evaluation of our tool and used the judgment of the software developers as an alternative to an objective ground truth. We illustrate the complete evaluation process in Figure 4.

5.1 Quantitative Evaluation Results

We conducted nine isolated test runs. Each test run consisted of a fresh instance of GERALLT. Each of these instances was given the same initial prompt as described in Section 4.2 and Table 1. Since the LLM-based components of GERALLT are inherently non-deterministic, each test run resulted in different judgments given by the evaluator component.

During these test runs, the controller performed 752 actions on RCE. After each action, the evaluator determined whether it deemed the effect these actions had on the GUI as problematic as described in Section 4.3 and in Table 2. The output of the evaluator stated a problematic result after 72 actions.

We then manually separated the outputs of the evaluator into true and false positives, i.e., into those that correctly describe the current state of the GUI and those that do not. This separation process resulted in seven true positive issues. We moreover determined two pairs of issues to result from the same underlying cause. Thus, the evaluation process results in five unique issues with the tool integration wizard of RCE. The problems range from optical problems to functional errors. We discussed the five issues with the development team of RCE who confirmed that they agreed with the classification given by our tool. Moreover, the developers agreed that these issues constituted "blind spots" in the existing testing process, i.e., that they did not consciously notice these issues during manual testing.

5.2 Example Issue

We show an example problem found by the evaluator in Figure 5. Here, the controller has opened the pop-up window asking the tool integrator to provide the *launch settings* of the tool to be integrated. These launch setting comprise a) the directory in which the tool is installed, b) the version of the tool, c) the absolute path to the working directory in which the tool shall be executed, and d) the maximal number of instances of the tool that

Table 2: Composition of the evaluator prompt. Newlines omitted where possible.

Role	You are a very experienced GUI Tester. You observe the GUI of a Software. A system performs actions on the GUI. After each action taken on the GUI you evaluate whether the software behaves as expected or if there are any issues. You are provided with a screenshot of the GUI before and after the action.
Task	Check if there are any inconsistencies, unexpected behaviour or UI Elements that are not visible. In detail check the following: - the size, position, height, width of the visual elements - Checking the message displayed, frequency and content - Checking alignment of radio buttons, drop downs - Verifying the title of each section and their correctness - Cross-checking the colors and its synchronization with the theme
Output	Format your output as JSON. For example: { "state": "problem", "reason": "The Text of the Description is not fully visible." } If there are no problems output: { "state": "okay" }
Action	The action performed between the two images was was: Click the 'Next ⟩' button to proceed to the next page of the Tool Integration Wizard and continue testing the remaining pages and their functionalities.
Closing	Are there any problems with the GUI after the action was performed?

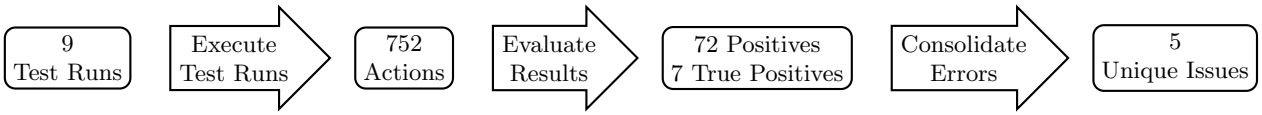
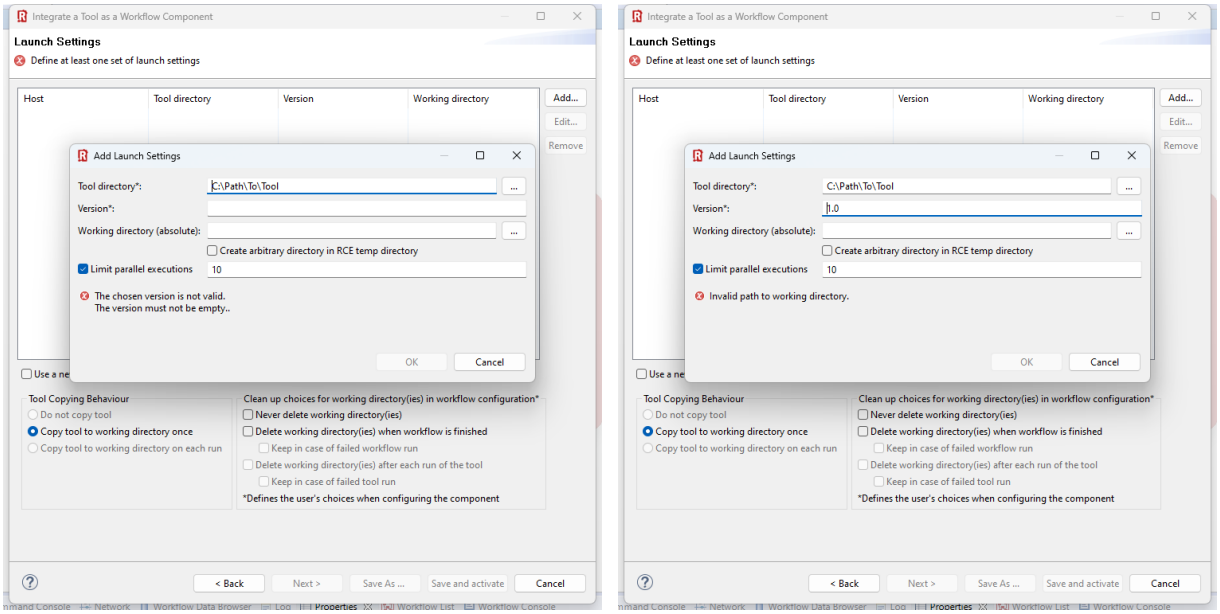


Fig. 4: The evaluation process for our system.



(a) Screenshot before the performed action

(b) Screenshot after the performed action

Fig. 5: Example error, where the evaluator criticized that the error message is too imprecise.

can be executed in parallel. The controller has moreover entered a path to the tool directory. The validation of the contents of the pop-up window proceeds from top to bottom. Hence, RCE informs the user about a problem with the empty field for the tool version, namely that "The chosen version is not valid. The version must not be empty..[sic]"

During the next iteration of GERALLT, the controller inputs the string "1.0" into the field "Version". Hence, RCE determines the supplied version to be valid. Since input validation proceeds with the next field in the form, RCE now informs the user that the (non-existent) path to the working directory is invalid.

The evaluator agent determines that this behavior is problematic and returns the explanation "The error message 'Invalid path to working directory' is visible even though no path was provided, which is inconsistent unless a path was entered." In other terms, the evaluator criticizes that a non-existent path cannot be invalid. Moreover, it criticizes that in the case of an empty version field, the user was informed about the field being empty (cf. Figure 5a), while they are given no such specific error message in the case of an empty working directory field (cf. Figure 5b).

5.3 Known Limitations

In the previous sections, we have presented the capabilities of GERALLT. We now turn our attention to discussing its limitations, particularly in terms of the system requirements of the implementation, the system under test, and the software qualities tested by GERALLT.

The current implementation of our system is limited to testing RCE on Windows. This is due to the choice of PyWinAuto [29] for the implementation of the GUI parser, which only allows automation of the Windows GUI. GNOME- or KDE-based GUIs could be tested using, e.g., Dogtail [30] or Appium [31,32]. This would require some implementation effort, but no change to the overall system architecture presented in this work.

In this work, we use GERALLT to test the tool integration wizard of RCE. This requires the user to perform left clicks and text inputs. Other features of RCE require more advanced interactions such as, e.g., dragging and dropping interface elements. Similarly to the previous case, implementing such interactions would require adaptations to the Output Parser as well as the Action Executor. However, no conceptual change of the architecture presented in Figure 2 would be required.

Finally, GERALLT only aims to find GUI behavior that human users would deem unintuitive and functional errors. In particular, we have not constructed the system to determine, e.g., accessibility or security issues exhibited by the system under test. Detecting such issues is out of the scope of this work. Moreover, in our opinion, such measures of quality are better tested for using rule-based, deterministic approaches.

6 Conclusion and Future Work

In this work we have given preliminary evidence that an LLM-based automated testing system can support software developers in exploratory GUI testing. Our developed system called GERALLT aims to determine behavior of the GUI that is deemed unintuitive or surprising by software developers. Moreover, the issues identified by GERALLT can find issues in "blind spots" of the software developers. Our work thus provides an interesting new direction for circumventing such blind spots in exploratory testing.

Our work constitutes a promising contribution to the state of the art. There are numerous avenues in which future research can continue. Our main aim in these future research directions is to evaluate GERALLT against existing manual testing processes. This requires objective measures of the efficiency and effectiveness of exploratory testing.

Moreover, we are looking to increase the applicability of GERALLT. For this, we are following three major directions: Making the system applicable to other features of RCE, using it on the same software running on additional operating systems, and testing it on other engineering software products. As discussed in Section 5.3, these extensions mainly amount to additional work on the implementation instead of significant changes to the architecture presented in Section 4.1.

We believe that our contribution in this work serves as an important stepping stone towards the development of LLM-based testing systems. These systems will provide valuable feedback on the intuitiveness of the GUI to developers without laboriously involving human testers. Such systems will not replace human feedback, but instead augment the testing process. Eventually, the aim of developing these systems is to move acceptance testing ever further "left" in the development cycle, thus decreasing the cost of software development and increasing software quality.

References

1. T. Rosenbach, "Developing an Interface between an LLM and the GUI of RCE," Bachelor Thesis, DHBW Mannheim, 2024.

2. V. Garousi and J. Zhi, "A survey of software testing practices in canada," *Journal of Systems and Software*, vol. 86, no. 5, pp. 1354–1376, May 2013.
3. ISO, *ISO/IEC 25010:2023*, ISO/IEC Std., 2023. [Online]. Available: https://www.iso.org/standard/78176.html
4. R. Haas, D. Elsner, E. Juergens, A. Pretschner, and S. Apel, "How can manual testing processes be optimized? developer survey, optimization guidelines, and case studies," in *ESEC/FSE*, ser. ESEC/FSE '21. ACM, Aug. 2021.
5. Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions," in *ICSE*, ser. ICSE '24. ACM, Apr. 2024, pp. 1–13.
6. D. Zimmermann and A. Koziolek, "Gui-based software testing: An automated approach using gpt-4 and selenium webdriver," in *A-TEST*. IEEE, Sep. 2023, pp. 171–174.
7. S. Bergsmann, A. Schmidt, S. Fischer, and R. Ramler, "First experiments on automated execution of gherkin test specifications with collaborating llm agents," in *A-TEST*, ser. A-TEST '24. ACM, Sep. 2024, pp. 12–15.
8. D. Zimmermann, P. Deubel, and A. Koziolek, "Evaluating the effectiveness of neuroevolution for automated gui-based software testing," in *ASEW*. IEEE, Sep. 2023, pp. 119–126.
9. C. Zhang, Y. Zheng, M. Bai, Y. Li, W. Ma, X. Xie, Y. Li, L. Sun, and Y. Liu, "How effective are they? exploring large language model based fuzz driver generation," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 1223–1235.
10. H. Guan, G. Bai, and Y. Liu, "Large language models can connect the dots: Exploring model optimization bugs with domain knowledge-aware prompts," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 1579–1591.
11. Z. Xue, L. Li, S. Tian, X. Chen, P. Li, L. Chen, T. Jiang, and M. Zhang, "Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 1643–1655.
12. P. Kaur, G. S. Kashyap, A. Kumar, M. T. Nafis, S. Kumar, and V. Shokeen, "From text to transformation: A comprehensive review of large language models' versatility," 2024. [Online]. Available: https://arxiv.org/abs/2402.16142
13. B. García, M. Leotta, F. Ricca, and J. Whitehead, "Use of chatgpt as an assistant in the end-to-end test script generation for android apps," in *A-TEST*, ser. A-TEST '24. ACM, Sep. 2024, pp. 5–11.
14. Z. Tian, H. Shu, D. Wang, X. Cao, Y. Kamei, and J. Chen, "Large language models for equivalent mutant detection: How far are we?" in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 1733–1745.
15. C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 819–831.
16. B. Yang, H. Tian, W. Pian, H. Yu, H. Wang, J. Klein, T. F. Bissyandé, and S. Jin, "Cref: An llm-based conversational software repair framework for programming tutors," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 882–894.
17. S. Shan, Y. Huo, Y. Su, Y. Li, D. Li, and Z. Zheng, "Face it yourselves: An llm-based two-stage strategy to localize configuration errors via logs," in *ISSTA*, ser. ISSTA '24. ACM, Sep. 2024, pp. 13–25.
18. T. Fulcini, G. Garaccione, R. Coppola, L. Ardito, and M. Torchiano, "Guidelines for gui testing maintenance: a linter for test smell detection," in *A-TEST*, ser. A-TEST '22. ACM, Nov. 2022.
19. T. Wong, "Chouette: An automated cross-platform ui crawler for improving app quality," in *ASWE*. IEEE, Sep. 2023, pp. 175–178.
20. R. Gu and J. M. Rojas, "An empirical study on the adoption of scripted gui testing for android apps," in *ASEW*. IEEE, Sep. 2023, pp. 179–182.
21. D. Gao, L. Ji, Z. Bai, M. Ouyang, P. Li, D. Mao, Q. Wu, W. Zhang, P. Wang, X. Guo, H. Wang, L. Zhou, and M. Z. Shou, "ASSISTGUI: Task-Oriented Desktop Graphical User Interface Automation," Dec. 2023.
22. B. Boden, J. Flink, N. Först, R. Mischke, K. Schaffert, A. Weinert, A. Wohlan, and A. Schreiber, "Rce: An integration environment for engineering and science," *SoftwareX*, vol. 15, p. 100759, Jul. 2021.
23. J. Flink, R. Mischke, K. Schaffert, D. Schneider, and A. Weinert, "Orchestrating tool chains for model-based systems engineering with rce," in *AERO*. IEEE, Mar. 2022, pp. 1–9.
24. R. Mischke, K. Schaffert, D. Schneider, and A. Weinert, "Automated and manual testing in the development of the research software rce," in *Computational Science – ICCS 2022*, D. Groen, C. de Mulatier, M. Paszynski, V. V. Krzhizhanovskaya, J. J. Dongarra, and P. M. A. Sloot, Eds. Cham: Springer International Publishing, 2022, pp. 531–544.
25. S. Peckham, J. Day, and DianaHbr, "Getting started with LLM prompt engineering," https://learn.microsoft.com/en-us/ai/playbook/technology-guidance/generative-ai/working-with-llms/prompt-engineering, accessed December 13, 2024.
26. pywinauto, "GitHub - pywinauto/pywinauto: Windows GUI automation with python (based on text properties)," Oct. 2019.
27. "Gerallt," https://github.com/DLR-SC/GERALLT.
28. OpenAI, "ChatGPT," Available at chat.openai.com.
29. PyWinAuto Contributors, "Pywinauto," Available at https://github.com/pywinauto/pywinauto.
30. "Dogtail," https://gitlab.com/dogtail/dogtail/.
31. "Selenium," https://www.selenium.dev/.
32. "Appium," https://develop.kde.org/docs/apps/tests/appium/.